

AI Prompt Trace

Prompt 1 – Solution Planning

You are a Senior Full Stack Engineer. Help me design a production-quality MVP Personal Finance Tracker that can realistically be completed within a strict 2-hour development window.

Requirements

- Frontend: React (Vite)
- Backend: Spring Boot 3 (Java 17)
- Database: MySQL
- State Management: Redux Toolkit
- Charts: Chart.js
- Authentication: JWT (preferred over OAuth due to time constraints)

Objective

Prioritize business value over feature count while keeping the architecture clean, modular, and scalable.

For every technical decision:

- Explain why it was chosen.
- Mention alternatives considered.
- Explain why the alternative was rejected.

Return:

- Folder structure
- Database schema
- API design
- Feature prioritization
- Development timeline

Prompt 2 – Feature Prioritization

Act as a Senior Product Engineer.

Given only two hours, identify which features should be implemented first.

Evaluate each feature based on:

- Business value
- User impact

- Development effort
- Risk
- Time required

Produce a prioritization matrix with Must Have, Should Have, Could Have, and Won't Have categories and justify every decision.

Prompt 3 – Backend Architecture

Generate a production-ready Spring Boot backend using Java 17.

Requirements:

- Layered Architecture
- Controller → Service → Repository
- DTO pattern
- Exception handling
- Validation
- Spring Security
- JWT Authentication
- Lombok
- JPA
- MySQL

Use clean coding practices and SOLID principles.

Prompt 4 – Database Design

Design an optimized relational database for a finance tracker.

Requirements:

- User
- Transaction
- Budget

Use:

- BigDecimal for monetary values
- Proper indexes
- Foreign keys
- Constraints
- Meaningful table names

Explain normalization decisions.

Prompt 5 – Authentication Module

Generate a secure authentication module using Spring Security and JWT.

Requirements:

- Registration
- Login
- Password hashing with BCrypt
- Stateless authentication
- JWT Filter
- SecurityConfig
- Token generation and validation

Ensure APIs follow REST standards.

Prompt 6 – Transaction Module

Generate a complete CRUD module for transactions.

Requirements:

- Add transaction
- Update transaction
- Delete transaction
- Get all transactions
- Filter by logged-in user
- Validation
- DTO mapping

Use clean service-layer architecture.

Prompt 7 – Budget Module

Create a budget management module.

Requirements:

- Monthly budget
- Category-wise budget
- Duplicate budget validation
- CRUD APIs
- Proper DTOs

Return production-ready code.

Prompt 8 – Dashboard Module

Create a dashboard service that calculates:

- Total Income
- Total Expense
- Balance
- Remaining Budget
- Budget vs Actual
- Expense by Category

Optimize calculations and use BigDecimal for financial accuracy.

Prompt 9 – Security Review

Review the backend architecture and identify:

- Security issues
- Performance improvements
- Scalability improvements
- Best practices
- Code smells

Provide recommendations before implementation.

Prompt 10 – Frontend Architecture

Design a scalable React architecture.

Requirements:

- Vite
- Redux Toolkit
- React Router
- Axios
- Material UI
- Chart.js

Organize the project using feature-based folders.

Explain why Redux Toolkit was selected instead of Context API.

Prompt 11 – API Integration

Generate an Axios service layer.

Requirements:

- Base URL configuration
- JWT interceptor
- Error handling
- Reusable API methods
- Environment variables

Prompt 12 – Redux Toolkit

Generate Redux Toolkit slices for:

- Authentication
- Transactions
- Budgets
- Dashboard

Use createAsyncThunk for API integration.

Include loading, success, and error states.

Prompt 13 – Dashboard UI

Build a responsive dashboard.

Include:

- Summary cards
- Expense Pie Chart
- Budget vs Actual table
- Remaining Budget
- Responsive Grid Layout

Use Material UI and Chart.js.

Prompt 14 – Transaction UI

Build a complete transaction management page.

Requirements:

- Add
 - Edit
 - Delete
 - Search
 - Category Filter
 - Responsive table
 - Form validation
-

Prompt 15 – Budget UI

Build a budget management page.

Requirements:

- Monthly budgets
 - Category selection
 - CRUD operations
 - Month filter
 - Responsive layout
-

Prompt 16 – Code Review

Review the complete React + Spring Boot project.

Check for:

- REST API consistency
- Naming conventions
- Exception handling
- JWT flow
- DTO mapping
- Folder organization
- SOLID principles
- Performance improvements

Provide refactoring suggestions where necessary.

Prompt 17 – Documentation

Generate professional project documentation.

Include:

- Architecture
- Folder structure
- Technology choices
- API documentation
- Database schema
- Setup instructions
- Trade-offs
- Future enhancements

Write the documentation as if it were for a GitHub repository used in a technical interview.

Prompt 18 – Final Validation

Act as a Senior Technical Interviewer.

Review this project against the assignment requirements.

Evaluate:

- Code quality
- Architecture
- Feature completeness
- Maintainability
- Scalability
- UI/UX
- Documentation

Suggest improvements that could increase the evaluation score.